



**Corso di Laurea Magistrale in Informatica**

# **Investigating the Role of Unit Tests for Performance Regression Testing Optimization**

**Prof. Dario Di Nucci**  
**Dott. Antonio Trovato**

**Daniele Fabiano**  
**Mat.: 0522501738**



# Introduzione e Background

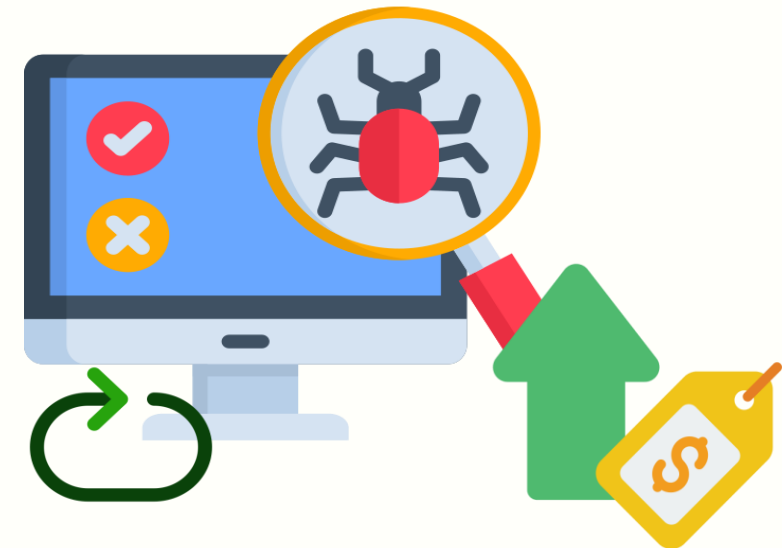


Quality Assurance

# Introduzione e Background



Quality Assurance



Regression Testing

## Regression Test Suite Optimization (RTSO)



Test Suite Minimization (TSM)



Test Case Selection (TCS)



Test Case Prioritization (TCP)

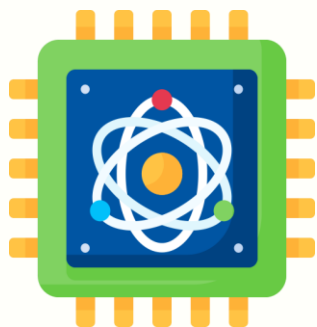
## Algoritmi State of the Art (SOTA) per RTSO - TCS:



Additional Greedy di Yoo and Harman [1]



DIV-GA di Panichella et al. [2]



IGDec-QAOA di Wang et al. [3]

QAOA-TCS di Trovato et al. [4]

[1] Yoo, S., & Harman, M. (2007, July). Pareto efficient multi-objective test case selection. In *Proceedings of the 2007 international symposium on Software testing and analysis* (pp. 140-150).

[2] Panichella, A., Oliveto, R., Di Penta, M., & De Lucia, A. (2014). Improving multi-objective test case selection by injecting diversity in genetic algorithms. *IEEE Transactions on Software Engineering*, 41(4), 358-383.

[3] Wang, X., Ali, S., Yue, T., & Arcaini, P. (2024). Quantum approximate optimization algorithm for test case optimization. *IEEE Transactions on Software Engineering* 50(12), 3249-3264.

[4] Trovato, A., Beseda, M., & Di Nucci, D. (2025, June). A preliminary investigation on the usage of quantum approximate optimization algorithms for test case selection. In *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion* (pp. 56-60).

Empirical Software Engineering (2021) 26: 133  
<https://doi.org/10.1007/s10664-021-10037-x>



## Applying test case prioritization to software microbenchmarks

Christoph Laaber<sup>1</sup>  · Harald C. Gall<sup>1</sup>  · Philipp Leitner<sup>2</sup> 

Accepted: 16 August 2021 / Published online: 30 September 2021  
© The Author(s) 2021

### Abstract

Regression testing comprises techniques which are applied during software evolution to uncover faults effectively and efficiently. While regression testing is widely studied for functional tests, performance regression testing, e.g., with software microbenchmarks, is hardly investigated. Applying test case prioritization (TCP), a regression testing technique, to software microbenchmarks may help capturing large performance regressions sooner

Empirical Software Engineering (2021) 26: 133  
<https://doi.org/10.1007/s10664-021-10037-x>



## Applying test case prioritization to software microbenchmarks

Christoph Laaber<sup>1</sup>  · Harald C. Gall<sup>1</sup>  · Philipp Leitner<sup>2</sup> 

**Algoritmo (Additional) Greedy  
Coverage a livello di metodo  
TCP a doppia prioritizzazione**



UNIVERSITY OF SALERNO  
Computer Science Department  
Master Degree

GRADUATION THESIS

## Investigating the Role of Unit Tests in Performance Regression Testing Optimization





UNIVERSITY OF SALERNO

Computer Science Department

## **TCS multi-obiettivo**

## **Varietà degli algoritmi scelti**

## **Generazione di casi di test**

## Identifying and Replicating Code Patterns Driving Performance Regressions in Software Systems

Denivan Campos<sup>\*</sup>, Luana Martins<sup>†</sup>, Emanuela Guglielmi<sup>\*</sup>, Michele Tucci<sup>‡</sup>,  
Daniele Di Pompeo<sup>‡</sup>, Simone Scalabrino<sup>\*</sup>, Vittorio Cortellessa<sup>‡</sup>, Dario Di Nucci<sup>†</sup>, Rocco Oliveto<sup>\*</sup>  
denivan.dasilva@unimol.it, lalmeida.martins@unisa.it, emanuela.guglielmi@unimol.it, michele.tucci@univaq.it,  
daniele.dipompeo@univaq.it, simone.scalabrino@unimol.it, vittorio.cortellessa@univaq.it, ddinucci@unisa.it,  
rocco.oliveto@unimol.it

<sup>\*</sup>University of Molise, Pesche, Italy

<sup>†</sup>University of Salerno, Salerno, Italy

<sup>‡</sup>University of L'Aquila, L'Aquila, Italy

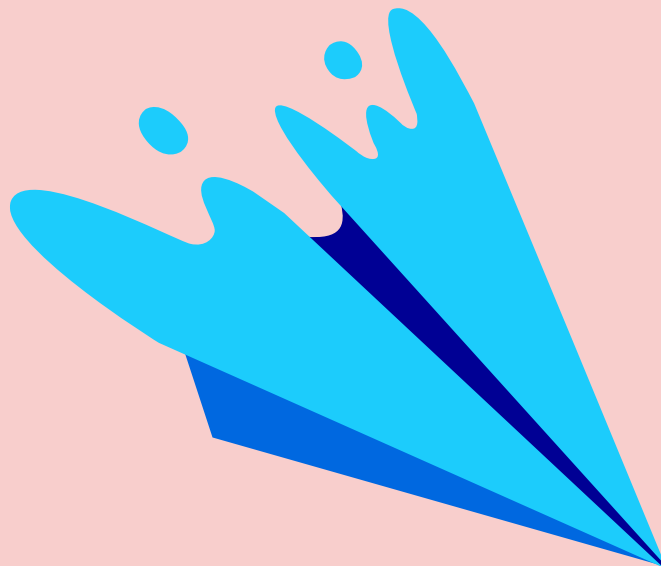
**Abstract—Context:** Performance regressions negatively impact execution time and memory usage of software systems. Nevertheless, there is a lack of systematic methods to evaluate the effectiveness of performance test suites. Performance mutation testing, which introduces intentional defects (*mutants*) to measure and enhance fault-detection capabilities, is promising but under-explored. A key challenge is understanding if generated mutants accurately reflect real-world performance issues. **Goal:** This study evaluates and extends mutation operators for performance testing. Its objectives include (i) collecting existing performance mutation operators, (ii) introducing new operators from real-world code changes that impact performance, and (iii) evaluating

tems operate without issues [7]–[12]. Despite its importance, there remains a significant gap in systematic methods for evaluating the effectiveness of performance test suites [12], [13].

In this regard, performance mutation testing offers a promising approach. Like functional mutation testing, performance mutation consists of introducing intentional performance issues (*mutants*) to assess the detection capabilities of performance tests. Sánchez et al. [6], [14] and Delgado-Pérez et al. [3] explore the possible benefits and limitations of applying mutation testing for performance and define per-

## Identifying and Replicating Code Patterns Driving Performance Regressions in Software Systems

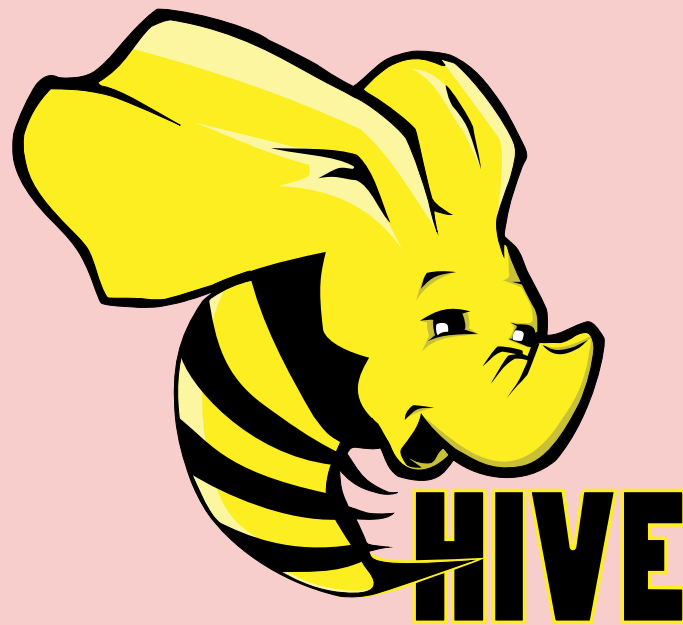
Denivan Campos<sup>\*</sup> Luana Martins<sup>†</sup>, Emanuela Guglielmi<sup>\*</sup>, Michele Tucci<sup>‡</sup>,  
Daniele Di Pompeo<sup>‡</sup>, Simone Scalabrino<sup>\*</sup>, Vittorio Cortellessa<sup>‡</sup>, Dario Di Nucci<sup>†</sup>, Rocco Oliveto <sup>\*</sup>  
denivan.dasilva@unimol.it, lalmeida.martins@unisa.it, emanuela.guglielmi@unimol.it, michele.tucci@univaq.it,  
daniele.dipompeo@univaq.it, simone.scalabrino@unimol.it, vittorio.cortellessa@univaq.it, ddinucci@unisa.it,  
rocco.oliveto@unimol.it



| Pre-fix commit - cd91695168f9edaac8c1f0140f4a023258f265d4  |                 |
|------------------------------------------------------------|-----------------|
| Class Full Qualified Name                                  | Lines of code   |
| org.apache.avro.data.RecordBuilderBase                     | 76, 77          |
| org.apache.avro.util.Utf8                                  | 64, 65, 116-118 |
| Post-fix commit - ff5c4d6db9af3edd915a2176a627d51eb3478da3 |                 |
| Class Full Qualified Name                                  | Lines of code   |
| org.apache.avro.data.RecordBuilderBase                     | 76              |
| org.apache.avro.util.Utf8                                  | 65, 116         |

## Identifying and Replicating Code Patterns Driving Performance Regressions in Software Systems

Denivan Campos<sup>\*</sup> Luana Martins<sup>†</sup>, Emanuela Guglielmi<sup>\*</sup>, Michele Tucci<sup>‡</sup>,  
Daniele Di Pompeo<sup>‡</sup>, Simone Scalabrino<sup>\*</sup>, Vittorio Cortellessa<sup>‡</sup>, Dario Di Nucci<sup>†</sup>, Rocco Oliveto <sup>\*</sup>  
denivan.dasilva@unimol.it, lalmeida.martins@unisa.it, emanuela.guglielmi@unimol.it, michele.tucci@univaq.it,  
daniele.dipompeo@univaq.it, simone.scalabrino@unimol.it, vittorio.cortellessa@univaq.it, ddinucci@unisa.it,  
rocco.oliveto@unimol.it



| Pre-fix commit - 186fb0d85af63b61bc10ba5372e35895754b1a6a  |                  |
|------------------------------------------------------------|------------------|
| Class Full Qualified Name                                  | Lines of code    |
| org.apache.hadoop.hive.metastore.HiveMetaStoreClient       | 565-572, 575     |
| org.apache.hadoop.hive.metastore.utils.MetaStoreUtils      | 802-813, 815-824 |
| Post-fix commit - 8863468898c9242fc334b9c93bf79e249e53842d |                  |
| Class Full Qualified Name                                  | Lines of code    |
| org.apache.hadoop.hive.metastore.HiveMetaStoreClient       | 567-581          |
| org.apache.hadoop.hive.metastore.utils.MetaStoreUtils      | 802-825          |



## Automated Generation and Evaluation of JMH Microbenchmark Suites From Unit Tests

Mostafa Jangali<sup>ID</sup>, Yiming Tang<sup>ID</sup>, Niclas Alexandersson<sup>ID</sup>, Philipp Leitner<sup>ID</sup>, Jinqiu Yang, and Weiyi Shang

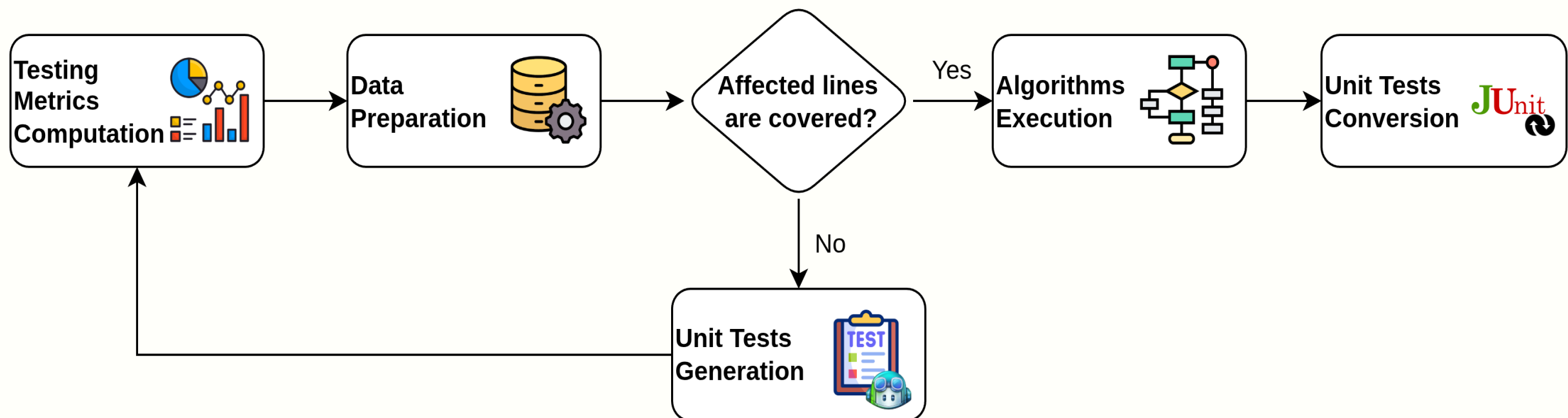
**Abstract**—Performance is a crucial non-functional requirement of many software systems. Despite the widespread use of performance testing, developers still struggle to construct and evaluate the quality of performance tests. To address these two major challenges, we implement a framework, dubbed *ju2jmh*, to automatically generate performance microbenchmarks from JUnit tests and use mutation testing to study the quality of generated microbenchmarks. Specifically, we compare our *ju2jmh* generated benchmarks to manually written JMH benchmarks and to automatically generated JMH benchmarks using the AutoJMH framework, as well as directly measuring system performance with JUnit tests. For this purpose, we have conducted a study on three subjects (Rxjava, Eclipse-collections, and Zipkin) with ~454K *source lines of code* (SLOC), 2,417 JMH benchmarks (including manually written and generated AutoJMH benchmarks) and 35,084 JUnit tests. Our results show that the *ju2jmh* generated JMH benchmarks consistently outperform using the execution time and throughput of JUnit tests as a proxy of performance and JMH benchmarks automatically generated using the AutoJMH framework while being comparable to JMH benchmarks manually written by developers in terms of tests' stability and ability to detect performance bugs. Nevertheless, *ju2jmh* benchmarks are able to cover more of the software applications than manually written JMH benchmarks during the microbenchmark execution. Furthermore, *ju2jmh* benchmarks are generated automatically, while manually written JMH benchmarks require many hours of hard work and attention; therefore our study can reduce developers' effort to construct microbenchmarks. In addition, we identify three factors (too low test workload, unstable tests and limited mutant coverage) that affect a benchmark's ability to detect performance bugs. To the best of our knowledge, this is the first study aimed at assisting developers in fully automated microbenchmark creation and assessing microbenchmark quality for performance testing.

## Automated Generation and Evaluation of JMH Microbenchmark Suites From Unit Tests

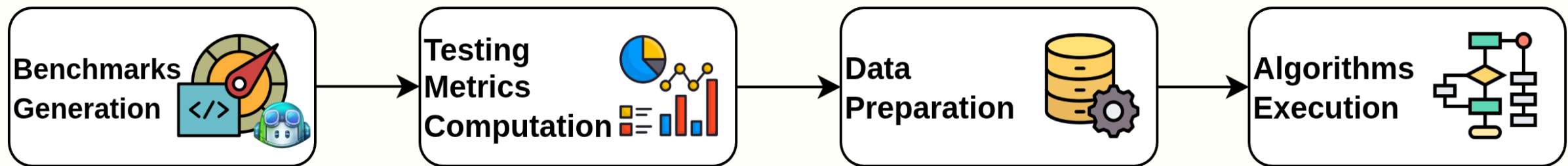
Mostafa Jangali<sup>ID</sup>, Yiming Tang<sup>ID</sup>, Niclas Alexandersson<sup>ID</sup>, Philipp Leitner<sup>ID</sup>, Jinqiu Yang, and Weiyi Shang

**JUnit**to**JMH**

# Upstream Method Approach



# Downstream Method Approach





Q. **RQ1:** Qual è il miglior algoritmo per selezionare un sottoinsieme di microbenchmark?

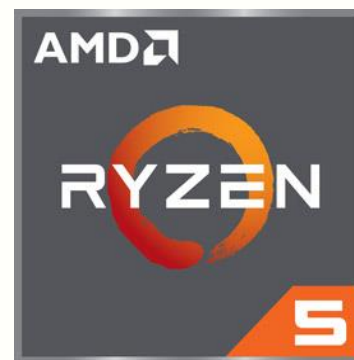
Numero di esecuzioni degli algoritmi: 10 per pre-fix e 10 post-fix

Output degli algoritmi: Insieme di soluzioni come fronte di Pareto

Statistiche calcolate: Media Campionaria e Deviazione Standard

Test Statistici: Shapiro-Wilk, Kruskal-Wallis H, Dunn, Effect Size

Specifiche della macchina:



🔍 **RQ1:** Qual è il miglior algoritmo per selezionare un sottoinsieme di microbenchmark?

🔍 **RQ1.a:** Quale algoritmo è in grado di selezionare il sottoinsieme di microbenchmark più efficace?

| Sistema | Approccio  | Algoritmo     | Media        | Dev. Standard |
|---------|------------|---------------|--------------|---------------|
| Avro    | Upstream   | Add. Greedy   | 0.491        | 0.035         |
|         |            | <b>DIV-GA</b> | <b>0.991</b> | <b>0.027</b>  |
|         |            | IGDec-QAOA    | 0.325        | 0.038         |
|         |            | QAOA-TCS      | 0.406        | 0.151         |
|         | Downstream | Add. Greedy   | 0.658        | 0.020         |
|         |            | <b>DIV-GA</b> | <b>1.000</b> | <b>0.000</b>  |
|         |            | IGDec-QAOA    | 0.150        | 0.037         |
|         |            | QAOA-TCS      | 0.423        | 0.174         |

🔍 **RQ1:** Qual è il miglior algoritmo per selezionare un sottoinsieme di microbenchmark?

🔍 **RQ1.a:** Quale algoritmo è in grado di selezionare il sottoinsieme di microbenchmark più efficace?

| Sistema | Approccio  | Algoritmo     | Media        | Dev. Standard |
|---------|------------|---------------|--------------|---------------|
| Hive    | Upstream   | Add. Greedy   | 0.622        | 0.032         |
|         |            | <b>DIV-GA</b> | <b>0.641</b> | <b>0.117</b>  |
|         |            | IGDec-QAOA    | 0.137        | 0.072         |
|         |            | QAOA-TCS      | 0.290        | 0.174         |
|         | Downstream | Add. Greedy   | 0.556        | 0.046         |
|         |            | <b>DIV-GA</b> | <b>0.817</b> | <b>0.055</b>  |
|         |            | IGDec-QAOA    | 0.234        | 0.084         |
|         |            | QAOA-TCS      | 0.620        | 0.134         |

Q RQ1: Qual è il miglior algoritmo per selezionare un sottoinsieme di microbenchmark?

Q RQ1.b: Quale algoritmo è in grado di selezionare il sottoinsieme di microbenchmark più efficiente?

| Sistema | Approccio  | Algoritmo   | Media  | Dev. Standard |
|---------|------------|-------------|--------|---------------|
| Avro    | Upstream   | Add. Greedy | 0.0035 | 0.0002        |
|         |            | DIV-GA      | 0.0010 | 2.6492        |
|         |            | IGDec-QAOA  | 0.0014 | 7.2017        |
|         |            | QAOA-TCS    | 0.0010 | 0.0001        |
|         | Downstream | Add. Greedy | 0.0055 | 0.0007        |
|         |            | DIV-GA      | 0.0025 | 0.0007        |
|         |            | IGDec-QAOA  | 0.0019 | 0.0007        |
|         |            | QAOA-TCS    | 0.0016 | 0.0006        |

Q RQ1: Qual è il miglior algoritmo per selezionare un sottoinsieme di microbenchmark?

Q RQ1.b: Quale algoritmo è in grado di selezionare il sottoinsieme di microbenchmark più efficiente?

| Sistema | Approccio  | Algoritmo   | Media  | Dev. Standard |
|---------|------------|-------------|--------|---------------|
| Hive    | Upstream   | Add. Greedy | 0.0200 | 0.0007        |
|         |            | DIV-GA      | 0.0043 | 0.0001        |
|         |            | IGDec-QAOA  | 0.0016 | 0.0010        |
|         |            | QAOA-TCS    | 0.0041 | 0.0005        |
|         | Downstream | Add. Greedy | 0.1109 | 0.0006        |
|         |            | DIV-GA      | 0.0158 | 0.0014        |
|         |            | IGDec-QAOA  | 0.0041 | 0.0019        |
|         |            | QAOA-TCS    | 0.0100 | 0.0020        |

🔍 **RQ2:** Qual è il miglior approccio per creare e selezionare i microbenchmark?

Efficienza: DIV-GA è molto vicino agli algoritmi quantistici

Accesso: Complessità nell'usare macchine quantistiche e simulatori

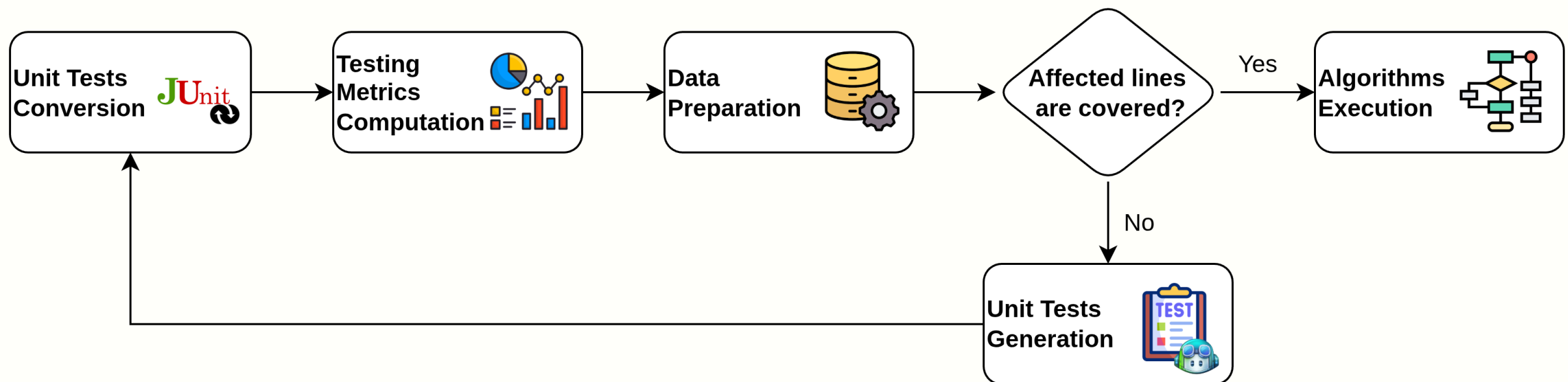
Upstream: Tempi di esecuzione bassi ma misurazioni meno precise

Downstream: Tempi di esecuzione più alti ma misurazioni più precise



# Come ridurre ulteriormente i costi?

## Modified Upstream Approach



## Introduzione e Background

seso<sup>lab</sup>  
SOFTWARE ENGINEERING  
SALERNO



Quality Assurance



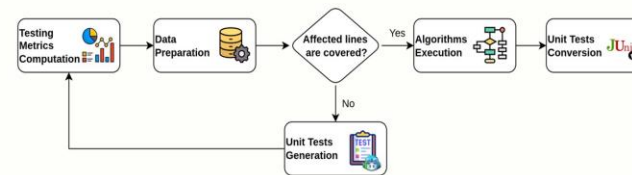
Regression Testing

d.fabiano3@studenti.unisa.it  
tensa53.github.io  
linkedin.com/in/danielefabiano53

Investigating the Role of Unit Tests for  
Performance Regression Testing  
Daniele Fabiano  
Università degli Studi di Salerno

## Upstream Method Approach

seso<sup>lab</sup>  
SOFTWARE ENGINEERING  
SALERNO

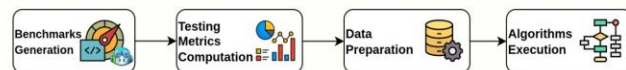


d.fabiano3@studenti.unisa.it  
tensa53.github.io  
linkedin.com/in/danielefabiano53

Investigating the Role of Unit Tests for  
Performance Regression Testing  
Daniele Fabiano  
Università degli Studi di Salerno

## Downstream Method Approach

seso<sup>lab</sup>  
SOFTWARE ENGINEERING  
SALERNO



d.fabiano3@studenti.unisa.it  
tensa53.github.io  
linkedin.com/in/danielefabiano53

Investigating the Role of Unit Tests for  
Performance Regression Testing  
Daniele Fabiano  
Università degli Studi di Salerno

## Studio Empirico

seso<sup>lab</sup>  
SOFTWARE ENGINEERING  
SALERNO

Q.RQ1: Qual è il miglior algoritmo per selezionare un sottoinsieme di microbenchmark?

Q.RQ1.a: Quale algoritmo è in grado di selezionare il sottoinsieme di microbenchmark più efficace?

| Sistema | Approccio  | Algoritmo   | Media | Dev. Standard |
|---------|------------|-------------|-------|---------------|
| Avro    | Upstream   | Add. Greedy | 0.491 | 0.035         |
|         |            | DIV-GA      | 0.991 | 0.027         |
|         |            | IGDec-QAOA  | 0.325 | 0.038         |
|         | Downstream | QAOA-TCS    | 0.406 | 0.151         |
|         |            | Add. Greedy | 0.658 | 0.020         |
|         |            | DIV-GA      | 1.000 | 0.000         |
|         |            | IGDec-QAOA  | 0.150 | 0.037         |
|         |            | QAOA-TCS    | 0.423 | 0.174         |

d.fabiano3@studenti.unisa.it  
tensa53.github.io  
linkedin.com/in/danielefabiano53

Investigating the Role of Unit Tests for  
Performance Regression Testing  
Daniele Fabiano  
Università degli Studi di Salerno

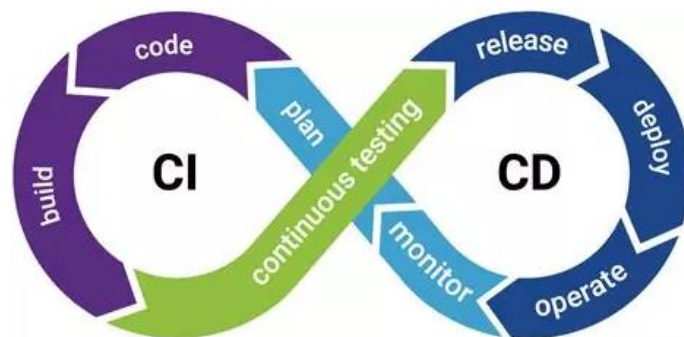
# Investigating the Role of Unit Tests for Performance Regression Testing

PDF della tesi



Grazie!

## Sviluppi Futuri



Implementare gli approcci in CI/CD

Daniele Fabiano

[d.fabiano3@studenti.unisa.it](mailto:d.fabiano3@studenti.unisa.it)

[tensa53.github.io](https://tensa53.github.io)

[linkedin.com/in/danielefabiano53](https://linkedin.com/in/danielefabiano53)